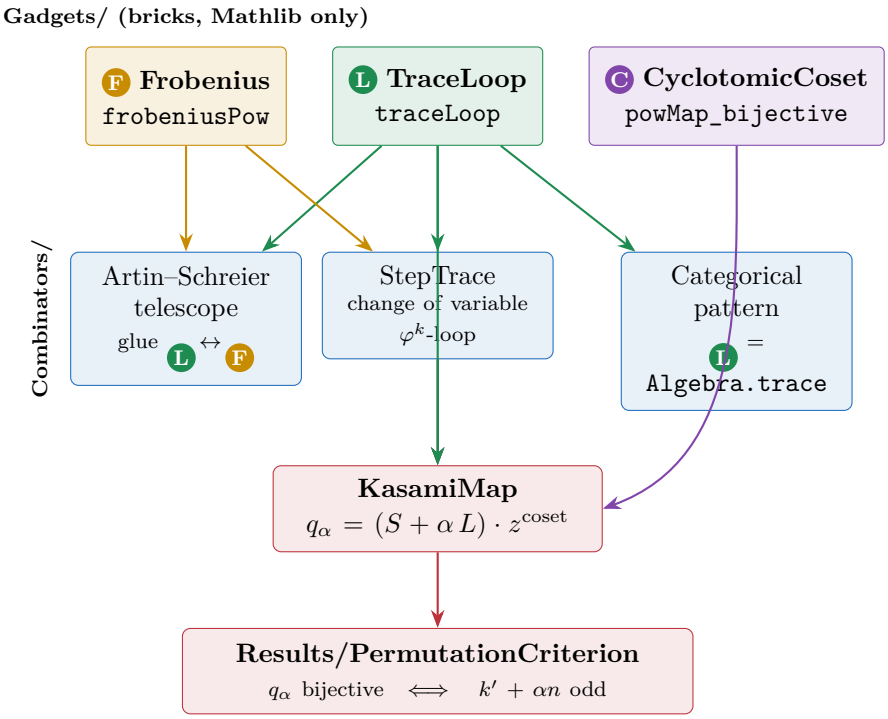
How the `Kasami` modules are built from the LEGO blocks

Bricks (L) (F) (C) → combinators → the Kasami map → the permutation criterion

A picture book of the `Kasami` Lean library (Lean / Mathlib) — `import Kasami`

What this note is. The `Kasami/` library is a block-structured re-presentation of the *permutation half* of Dobbertin's 1999 paper *Kasami Power Functions, Permutation Polynomials and Cyclic Difference Sets*. The design is deliberately LEGO-like: fix three primitive **bricks** (the maps (L) (F) (C)), add a few **combinators** that snap them together, then **assemble** the Kasami map and read off the headline permutation criterion by pure composition. Every box drawn below is a ✓ *sorry*-free Lean declaration (standard axioms `propext`, `Classical.choice`, `Quot.sound` only). This document describes the current `Kasami/` modules, not the older `Equation1/` extract (which is the reused engine underneath).

The architecture at a glance



The dependency edges are literal Lean `imports`. The (C) brick and (L) brick feed the assembly directly (the coset power and the αL term); the (F)/(L) combinators supply the numerator engine S and the telescoping identities that the underlying root-counting argument consumes.

1 The three bricks — `Kasami/Gadgets/`

Each brick depends on *Mathlib only*, so all three are reusable and upstreamable on their own.

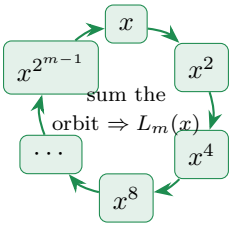
(F) Frobenius / doubling — `Gadgets/Frobenius.lean`

The inserted map $\varphi : x \mapsto x^2$ and its iterates $\varphi^r : x \mapsto x^{2^r}$ ("doubling on the exponent"). Closing law and cycling are the arithmetic that brick (C) later turns into orbit combinatorics.

```
def frobenius (x : F) : F := x ^ p          -- φ: x ↦ x^p  (p=2: doubling)
def frobeniusPow (r : ℕ) (x : F) : F := x ^ (p ^ r)   -- φ^r
frob_cycle           : x ^ Fintype.card F = x          -- closing law (Fermat) ✓
frobeniusPow_periodic: |F|=p^n ⇒ φ^r x = φ^(r % n) x   ✓
frobeniusPow_bijective: Function.Bijective (frobeniusPow p r)  ✓
```

L Linearized trace loop — `Gadgets/TraceLoop.lean`

The Frobenius map *closed into a loop*: $L_m(x) = \sum_{i<m} x^{2^i}$. Additive, Frobenius-equivariant, and at full length idempotent (a bit).




```
def traceLoop (m : ℕ) (x : F) : F := ∑_i<m x ^ (2 ^ i)
traceLoop_add      : traceLoop m (x+y) = traceLoop m x + traceLoop m y   ✓  -- additive
traceLoop_frobenius: traceLoop m (x^2) = (traceLoop m x)^2               ✓  -- equivariant
traceLoop_sq       : |F|=2^n ⇒ (traceLoop n x)^2 = traceLoop n x        ✓  -- idempotent
traceLoop_isBit    : |F|=2^n ⇒ traceLoop n x = 0 ∨ traceLoop n x = 1    ✓  -- a bit
```

C Cyclotomic-coset bookkeeping — `Gadgets/CyclotomicCoset.lean`

Where F cycles a *point*, C cycles an *exponent*: $\times 2$ permutes $\mathbb{Z}/(2^n-1)$, its orbits are the cyclotomic cosets. Two arithmetic facts and one permutation criterion are consumed downstream.

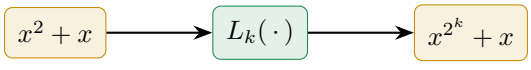
```
mersenne_coprime : Coprime k n ⇒ Coprime (2^k-1) (2^n-1)          ✓
inv_mod_exists   : Coprime k n ⇒ 1<2^n-1 ⇒ ∃b,(2^k-1)*b % (2^n-1)=1  ✓
powMap_bijective : Coprime (|F|-1) a ⇒ 0<a ⇒
                     Function.Bijective (fun x => x ^ a)           ✓
```

2 The combinators — `Kasami/Combinators/`

A combinator snaps bricks together. Three of them.

Artin–Schreier telescope — glue L to F

Inserting the doubling map into the loop makes it telescope: the loop turns the Artin–Schreier element $x^2 + x$ into a single Frobenius difference.



telescope: middle powers cancel

```
traceLoop_artin_schreier         : traceLoop k (x^2+x) = x^(2^k)+x        ✓
traceLoop_frobenius_invariant    : |F|=2^n ⇒ traceLoop n (x^(2^k)) = traceLoop n x  ✓
traceLoop_artin_schreier_zero    : |F|=2^n ⇒ traceLoop n (t^(2^k)+t) = 0    ✓
```

StepTrace — linearized change of variable

Insert $\varphi^k : x \mapsto x^{2^k}$ instead of φ and close *that* into a loop: the step-2^k partial trace $P_{k,k'}(x) = \sum_{j<k'} x^{2^{jk}}$. This is the **numerator engine** of the Kasami map.

```
def stepTrace (k k' : ℕ) (x : F) : F := ∑_j<k' x ^ (2 ^ (j*k))      -- P
def numeratorSum (k k' : ℕ) (x : F) : F := ∑_i=1..k' x ^ (2^(i*k)) -- S
stepTrace_add                  : stepTrace k k' (x+y) = ...x + ...y   ✓  -- linearized
numeratorSum_eq_stepTrace_frob: numeratorSum k k' x = (stepTrace k k' x)^(2^k)  ✓
stepTrace_telescope            : (stepTrace k k' x)^(2^k)+stepTrace k k' x
                                   = x^(2^(k'*k))+x                          ✓
```

Categorical trace pattern — L *is* the field trace

The loop is not ad hoc: it is Mathlib's `Algebra.trace`, the single "open → insert → close → keep what returns" move applied to the field's multiplication maps.

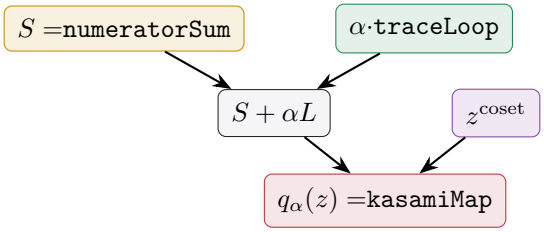
$$I \xrightarrow{\eta} V \otimes V^* \xrightarrow{m_x \otimes \mathrm{id}} V \otimes V^* \xrightarrow{\mathrm{swap}} V^* \otimes V \xrightarrow{\varepsilon} I$$

```
traceLoop_eq_algebraMap_trace :
  traceLoop (finrank (ZMod 2) L) x
    = algebraMap (ZMod 2) L (Algebra.trace (ZMod 2) L x)          ✓
```

3 The assembly — `Kasami/KasamiMap.lean`

The generalized Kasami polynomial is the three bricks snapped together:

$$q_\alpha(z) \;=\; \Big(\underbrace{S_{k,k'}(z)}_{\texttt{numeratorSum}\ \mathrm{F}/\mathrm{L}} + \alpha \cdot \underbrace{L_n(z)}_{\texttt{traceLoop}\ \mathrm{L}} \Big) \cdot z^{\overbrace{(2^n-1)-(2^k+1)}^{\texttt{coset power}\ \mathrm{C}}}.$$

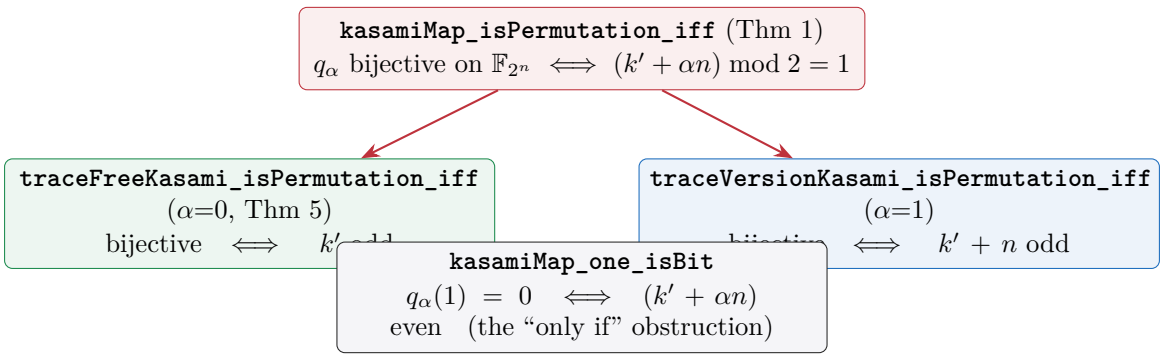



```
def kasamiMap (n k k' α : ℕ) (z : F) : F :=
  (numeratorSum k k' z + (α:F) * traceLoop n z) * z ^ (2^n - 1 - (2^k + 1))
kasamiMap_zero_pt : kasamiMap n k k' α 0 = 0      ✓  -- fixes 0
kasamiMap_eq_qKasami : kasamiMap n k k' α z = qKasami n k k' α z   ✓  -- rfl
```

The map is *definitionally* the paper's `qKasami`, so the permutation criterion proved for the underlying engine transfers verbatim.

4 The result — `Kasami/Results/PermutationCriterion.lean`

The headline (Dobbertin's Theorem 1), read off by composition:



```
theorem kasamiMap_isPermutation_iff (hn : |F|=2^n) (hk : k<n)
    (hcop : Coprime k n) (hk' : k*k' % n = 1 % n) (hk0 : 0<k)
    (hexp : 2^k+1 < 2^n-1) (α : ℕ) (hα : α=0 ∨ α=1) :
  Function.Bijective (kasamiMap n k k' α) ⟺ (k' + α*n) % 2 = 1    ✓
```

Verdict. Yes — the permutation half of Dobbertin's paper genuinely factors through this compositional toolkit. The three bricks L F C plus the three combinators (Artin–Schreier telescope, step-2^k change of variable, categorical pattern) assemble the Kasami map and deliver Theorem 1 and its two cases. The heavy root-counting argument they compose over lives in the reused, already *sorry*-free `Equation1/` engine; this `Kasami/` layer is its readable, block-structured re-presentation.
Scope. This covers the *permutation* half only (Theorem 1, Theorem 5, the trace-version case). The difference-set half (autocorrelation, 2-rank, the §3–4 conjecture) needs two further primitives not built here — a Fibonacci-style recursion block for the explicit inverse, and a character/Fourier block $\psi = (-1)^{\mathrm{Tr}}$ — whose hard content is not delivered by composition alone.